EPFO Learning Project - Beginner Notes (Setup to Validators)

Oct 29, 2026 · @Bhavesh

1. The big picture

We are building a small, working copy of a real bulk employee registration system, one tested piece at a time. By the end of Step 6 the foundation is finished: the app runs, talks to Oracle, runs a Spring Batch job, and checks Aadhaar, PAN and passport numbers.

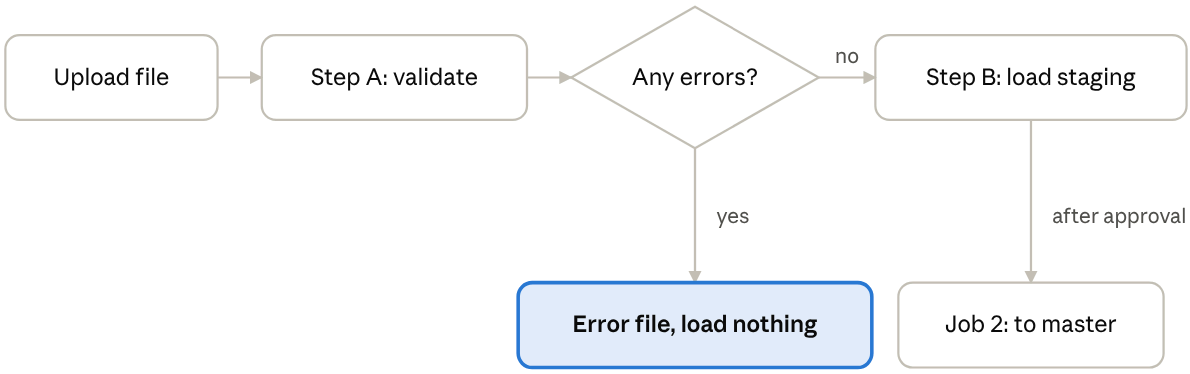
What the real system does. A user uploads a text file with one employee per line. The fields are separated by the marker `#~#`. The system checks every Aadhaar, PAN and passport number, stores good data in an Oracle database, and gives the user an error file when something is wrong.

One line of the file looks like this (Name, Mobile, Aadhaar, PAN, Passport, date of birth):

```
Rahul Sharma#~#9876543210#~#234567890124#~#ABCPE1234F#~#N1234567#~#1995-04-12
```

Our all-or-nothing design. Either every line is good and everything is loaded, or one bad line stops the whole load.

1. The user uploads the file through a REST endpoint (a web address that accepts the file).
2. Step A (validate) reads every line and collects all errors, not just the first one.
3. If there is any error, we write an error file (line number, field, reason) and load nothing.
4. If there are zero errors, Step B writes all rows into a staging table (a waiting area).
5. Later, an approval endpoint runs Job 2, which moves staging rows into the master table and rejects duplicate Aadhaar numbers.



all-or-nothing upload flow · 1 decision, 2 outcomes

Step A checks every line first. Only a clean file reaches the staging table, and Job 2 runs only after approval.

Why we build it this way.

- Small pieces: every step ends with a checkpoint you can run, so you always know what works.
- Same stack as the real project: plain Spring MVC 5.3.26 (no Spring Boot), Spring Batch 4.3.8, Oracle, Tomcat 9, `javax.*` packages and Java 17.
- No security, Tiles or Redis: they exist in the real project but would hide the lessons here.

Where we are today. Done: the environment, a hello web page, logging, the Oracle connection, a hello Batch job, and four validators with 19 passing tests. Next: reading the `#~#` file line by line (section 14 has the full roadmap).

2. The tools and what each one does

Six tools work together, and each has exactly one job. When something breaks, the first question is always: which tool's job failed?

Tool	Its one job	Where it lives on your PC
JDK 17.0.12	Turns `.java` files into `.class` files and runs Java programs	`C:\Program Files\jdk-17.0.12`

Tool	Its one job	Where it lives on your PC
Maven 3.9.16	Downloads libraries, calls the compiler, runs tests, packs the WAR file	`C:\maven\apache-maven`
Tomcat 9.0.123	A web server that unpacks and runs your WAR	`C:\tomcat9`
Oracle 26ai Free	The database that stores data; our service is `FREEPDB1` on port 1521	A Windows service
SQL*Plus	A text window where you type SQL commands into Oracle	The `sqlplus` command
VS Code	The editor where you write and save files	Installed app

Environment variables are named settings that programs read from Windows. We use two:

- `JAVA_HOME` points to the JDK folder. Maven and Tomcat read it to find Java.
- `CATALINA_HOME` points to the Tomcat folder (`C:\tomcat9`). Tomcat's start scripts read it. We set it with `set CATALINA_HOME=C:\tomcat9`, which lasts only for that Command Prompt window. After a restart or a new window, you set it again.

PATH is the list of folders Windows searches when you type a command such as `mvn`. Maven's folder is on it, so `mvn` works from any folder.

Why these particular versions. Tomcat 10 and later use `jakarta.*` package names. Spring 5.3 (our version, same as the real project) uses `javax.*`. They do not fit together, so we use Tomcat 9. Any Tomcat 10 copies on your Desktop or Downloads are unused.

Why space-free paths. Your Windows username contains a space (`vishal bhat`), which caused installer problems. So Maven, Tomcat and the JDK sit in simple paths like `C:\tomcat9`. Maven's download cache (`.m2`) is still under your user folder, which is why some commands need quotes around that path.

Two small facts worth remembering: the project is on drive `E:` (about 750 MB free, so keep it small), and Tomcat is on `C:`. That is why the build and deploy commands always use full paths.

3. The project structure: every folder and file

Maven expects a fixed folder layout, and putting a file in the wrong folder was the cause of one of our errors. The rule: production code goes under `src\main\java`, tests under `src\test\java`, and settings under `src\main\resources`.

```
E:\project\epfo-learn
  pom.xml                          Maven's recipe
  sql\schema-oracle10g.sql         Batch table script (copied from the Spring
Batch jar)
  src\main\java\com\epfo\learn\
    config\   AppInitializer, WebConfig, RootConfig, BatchConfig,
OracleBatchConfigurer
    web\      HelloController, DbCheckController, BatchTestController
    validation\  VerhoeffValidator, AadhaarValidator, PanValidator,
PassportValidator
  src\main\resources\log4j2.xml    Logging settings
  src\main\webapp\                 Web files folder (empty for now)
  src\test\java\com\epfo\learn\validation\   the 4 test classes
  target\                          Created by Maven; safe to delete

C:\tomcat9\conf\epfolearn.properties   Database login (outside the project,
on purpose)
```

File	Its job	Why it exists
`pom.xml`	Lists libraries (Spring, Batch, Oracle driver, JUnit) and build settings	Maven reads it to know what to download and how to build a WAR named `epfolearn`
`AppInitializer`	Starts Spring when Tomcat loads the app	It replaces the old `web.xml` file
`WebConfig`	Switches on Spring MVC and scans the `web` package for controllers	Web beans live here
`RootConfig`	Creates the database connection pool and `JdbcTemplate`	Shared beans live in the root context
`BatchConfig`	Switches on Spring Batch and defines the hello job	Where all Batch jobs will be defined

File	Its job	Why it exists
`OracleBatchConfigurer`	Builds Batch's record keeper with a safe isolation level	Fixes the Oracle `ORA-08177` error (section 9)
`HelloController`	Answers `/hello`	The first proof the app works
`DbCheckController`	Answers `/dbcheck` by asking Oracle who you are	Proves the database connection
`BatchTestController`	Answers `/runhello` by launching the hello job	Proves Batch works end to end
Four validators	Check Aadhaar, PAN, passport and the Verhoeff checksum	The rules our upload must enforce
Four test classes	Run the validators with known inputs	Fast proof that the rules are right
`log4j2.xml`	Says what to log and where	Controls the Tomcat console output
`epfolearn.properties`	Holds `db.url`, `db.user`, `db.password`	The password stays out of code and out of the WAR

Packages. Each Java file starts with a `package` line such as `package com.epfo.learn.validation;`. It must match the folder path under `src\main\java`. If it doesn't, the class is invisible to other classes.

4. From code to running app: build, copy, deploy

Your code only runs after three hand-offs: you write it, Maven packs it into a WAR, and Tomcat unpacks and runs that WAR. Nothing you type in VS Code reaches the browser until all three have happened.

What `mvn clean package` does. Maven runs a fixed chain of steps in order. Our output printed each one.

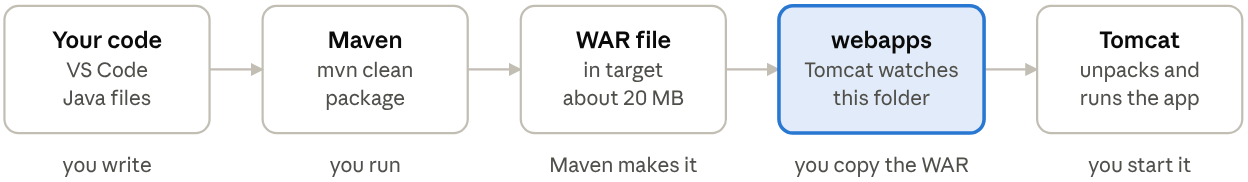
1. `clean` deletes the old `target` folder inside the project.
2. `resources` copies `src\main\resources` (for example `log4j2.xml`) to `target\classes`.
3. `compile` turns `.java` files into `.class` files in `target\classes`.
4. `testCompile` does the same for test code.

5. `test` (surefire) runs the unit tests. A failing test stops the build here.
6. `war` zips everything into `target\epfolearn.war`.

What a WAR is. A Web ARchive: a zip file with a fixed layout. Your own classes sit in `WEB-INF\classes`, and library jars (Spring, Batch, Oracle driver) sit in `WEB-INF\lib`. The servlet API is left out on purpose (scope `provided`), because Tomcat already has it. Our WAR is about 20 MB because of the libraries.

What Tomcat does with it. Tomcat watches its `webapps` folder. When it starts and finds `epfolearn.war`, it unzips it into a folder `webapps\epfolearn`. The file name becomes the web address path, so the app answers at `http://localhost:8080/epfolearn/`.

Nothing runs until the WAR reaches the webapps folder



build and deploy flow · 5 stages

The copy into `webapps` is the step that went missing twice, which is why it is highlighted.

The safe redeploy order. Stop Tomcat first, because a running Tomcat can lock files.

1. Stop Tomcat (Ctrl+C in its window).
2. `mvn clean package`, and wait for `BUILD SUCCESS`.
3. `rmdir /s /q C:\tomcat9\webapps\epfolearn` (remove the old unpacked app).
4. `del C:\tomcat9\webapps\epfolearn.war` (remove the old WAR).
5. `copy target\epfolearn.war C:\tomcat9\webapps\` (hand over the new WAR).
6. `dir C:\tomcat9\webapps` and check that `epfolearn.war` is listed.
7. `set CATALINA_HOME=C:\tomcat9`, then `C:\tomcat9\bin\catalina.bat run`.

Why order matters. `mvn clean` only empties your project's `target` folder. It never touches Tomcat. Your own `rmdir` and `del` empty `webapps`, and the `copy` must come after them. Twice we got a 404 because `webapps` held no WAR at all, so Tomcat had nothing to serve. Running `dir C:\tomcat9\webapps` before opening the browser finds this in seconds.

Optional shortcut. A small script named `deploy.bat` in the project folder can run steps 2 to 6 in order and stop if the build fails. We discussed it but did not create it.

5. How Spring starts and answers a web request

Tomcat knows nothing about Spring. Spring gets control because it plugs itself into Tomcat's startup, and from then on it routes every web address to a Java method.

Spring, in one idea. Spring creates the objects your app needs (called beans) and hands them to whoever asks. You describe what to create in configuration classes, and Spring does the wiring. This is called dependency injection, and `@Autowired` is how a class asks for a bean.

Why there is no `web.xml`. `web.xml` is an old XML file that told Tomcat which servlets exist and which URLs they handle. Since Servlet 3.0, Tomcat can find that information from Java code instead. Spring's jar contains a hook that Tomcat runs at startup. The hook looks for a class implementing `WebApplicationInitializer`, and that class is your `AppInitializer`. This is why the pom has `failOnMissingWebXml=false`: it tells the WAR plugin that a missing `web.xml` is fine.

What `AppInitializer` declares.

- Root config classes: `RootConfig` and `BatchConfig`. These beans are shared by the whole app: the database pool, `JdbcTemplate`, the Batch job beans.
- Servlet config class: `WebConfig`. This holds web beans such as controllers.
- Mapping `"/"`: one servlet, called the `DispatcherServlet`, receives every web address.

Controllers can use root beans (for example `JdbcTemplate`). Root beans cannot see controllers. This is the reason the database settings sit in the root config.

The annotations you have met.

Annotation	Meaning in plain words
`@Configuration`	This class defines beans
`@Bean`	The method's return value is a bean Spring should create and keep
`@EnableWebMvc`	Switch on Spring's web features (mapping addresses to methods)

Annotation	Meaning in plain words
`@ComponentScan("com.epfo.learn.web")`	Look in this package for classes Spring should manage
`@RestController`	This class answers web requests, and its return value is the response text
`@GetMapping("/hello")`	This method answers GET requests to `/hello`
`@Autowired`	Give me the bean of this type
`@EnableBatchProcessing`	Create the Spring Batch core beans (section 8)

The journey of one request, `/epfolearn/hello`.

1. The browser sends `GET /epfolearn/hello` to port 8080.
2. Tomcat sees the first part `/epfolearn`, so it hands the request to your app with the path `/hello`.
3. The `DispatcherServlet` finds the method marked `@GetMapping("/hello")`.
4. `HelloController.hello()` runs, writes a log line, and returns the text.
5. Spring sends the text back, and the browser shows it.

One request, from the URL bar to the answer



request flow · 5 stops

The DispatcherServlet is highlighted because it is the one piece you never wrote, yet every request passes through it.

WAR versus JAR. Both are zip files with an agreed layout.

	JAR	WAR
Stands for	Java ARchive	Web ARchive
Holds	Classes and resources at the root	`WEB-INF\classes`, `WEB-INF\lib` and web files
Purpose	A library, or a program with a `main` method	A web application
Runs how	`java -jar`, or used as a dependency	Placed in a server such as Tomcat

Spring Boot usually builds a runnable JAR with a server inside. The real project instead builds a WAR for an external Tomcat, and so do we.

6. Logging with Log4j2

A logger prints messages like `System.out.println`, but you control them from a settings file instead of editing code. That control matters once an app runs on a server and you need to know what it did.

How we use it in code. Each class gets its own logger, and then calls a method named for the message's importance:

```
private static final Logger log =
LogManager.getLogger(HelloController.class);

log.debug("hello endpoint called");
log.debug("dbcheck connected as {}", user);
```

The `{}` is a placeholder that is filled with the value after it. The imports come from `org.apache.logging.log4j`.

Levels. From most detailed to most serious: `trace`, `debug`, `info`, `warn`, `error`. A setting such as `info` means "show info and everything more serious", so debug lines are hidden.

What `log4j2.xml` says. Log4j2 looks for a file with this name on the classpath at startup. Ours sets the following, in words:

- One appender (a destination) that prints to the console, which is the Tomcat window.
- The package `com.epfo.learn` logs at `debug`, with `additivity=false` so each line prints once, not twice.
- Everything else (Spring, Oracle driver) logs at `info`.

Why not just `println`.

`System.out.println`	Logger
Always prints	Can be turned down by level, with no code change
Only the text you typed	Adds time, level, class and thread name automatically
Only the console	Can also write to files that roll over by day or size
Only your own lines	Spring and Spring Batch also log through it, so you can turn them up when something is wrong

The thread name will matter later, when several threads process lines at once and you need to know which one did what.

A two-minute experiment. Add `log.info("info line")` and `log.debug("debug line")` to a controller. Change `com.epfo.learn` from `debug` to `info` in `log4j2.xml`, rebuild, redeploy and call the page. The debug line disappears and nothing else changes. That is the main thing `println` cannot do.

7. Connecting the app to Oracle

The app reaches Oracle through a connection pool, and the login details live in a file outside the project. Both choices have a reason, and both are the same ideas the real project uses.

The ideas.

- **JDBC** is Java's standard way to talk to a database. The Oracle JDBC driver (`ojdbc11` 23.2.0.0, a jar in our pom) is the Oracle-specific part.
- **DataSource** is the object your code asks for a database connection. Spring's JDBC and Batch classes all take one.

- **Connection pool.** Opening a connection to Oracle is slow. A pool (commons-dbcp2 `BasicDataSource`) opens a few at startup and lends them out. When code "closes" a connection, it goes back to the pool.
- **JdbcTemplate** is Spring's helper for running SQL. It opens, uses and closes connections and turns database errors into Spring exceptions.

The login file. We do not put the password in code, because code gets shared and copied into the WAR. Instead, Tomcat's folder holds a small file, `C:\tomcat9\conf\epfolearn.properties`, with three lines (the password is typed by you, never shared):

```
db.url=jdbc:oracle:thin:@//127.0.0.1:1521/FREEPDB1
db.user=myuser
db.password=<your password>
```

Save it as type "All files" in Notepad, or Windows will name it `.txt`.

The configuration class. `RootConfig` reads that file and builds the pool:

```
@Configuration
@PropertySource("file:${catalina.base}/conf/epfolearn.properties")
public class RootConfig {

    @Bean(destroyMethod = "close")
    public BasicDataSource dataSource(Environment env) {
        BasicDataSource ds = new BasicDataSource();
        ds.setDriverClassName("oracle.jdbc.OracleDriver");
        ds.setUrl(env.getRequiredProperty("db.url"));
        ds.setUsername(env.getRequiredProperty("db.user"));
        ds.setPassword(env.getRequiredProperty("db.password"));
        ds.setInitialSize(2);
        ds.setMaxTotal(10);
        return ds;
    }

    @Bean
    public JdbcTemplate jdbcTemplate(DataSource dataSource) {
        return new JdbcTemplate(dataSource);
    }
}
```

- `@PropertySource` loads the file into Spring's `Environment`. Tomcat sets `catalina.base` to `C:\tomcat9`, so the path resolves on your machine.
- `getRequiredProperty` stops the app at startup with a clear message if a key is missing.
- `setInitialSize(2)` opens two connections at startup, and `setMaxTotal(10)` caps the pool at ten.
- `destroyMethod = "close"` closes the pool when the app stops.

The check page. `DbCheckController` runs `SELECT USER FROM dual` through `JdbcTemplate` and prints the result. Opening `/epfolearn/dbcheck` showed `Connected as: MYUSER`. That one line proves the driver, pool, login file, network and Oracle service all work.

Connecting by hand. In a Command Prompt, `sqlplus myuser@127.0.0.1:1521/FREEPDB1` asks for your password. The part after the `@` is host, port and service name, the same facts as in `db.url`.

If it fails. `ORA-01017` means wrong password. `ORA-12514` means the service name is wrong. `Could not resolve placeholder` means the properties file path or name is wrong. `ORA-12541` after a reboot means the Oracle listener service is not running.

8. Spring Batch basics and the hello job

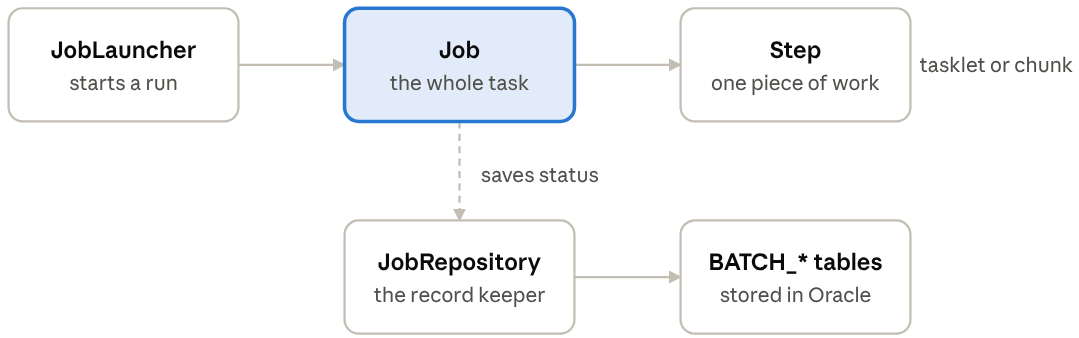
Spring Batch is a framework for jobs that process lots of records in a controlled, restartable way. Its biggest feature for us is that it records every run in database tables, so it knows what finished, what failed and where to resume.

Think of an assembly line. The Job is the whole line ("build a car"). A Step is one station on it ("weld", "paint", "inspect"). Switching the line on is one run. Each run is recorded.

Term	Plain meaning	Where you see it
Job	The named recipe: a name plus its steps. Defined once in Java	`helloJob` in `BatchConfig`
Step	One stage inside a job	`helloStep`
JobParameters	Inputs for one run, such as a file name or a timestamp	The `time` parameter in `/runhello`

Term	Plain meaning	Where you see it
JobInstance	The job plus one set of identifying parameters	A row in `BATCH_JOB_INSTANCE`
JobExecution	One attempt to run a job instance	A row in `BATCH_JOB_EXECUTION`
StepExecution	One attempt at one step	A row in `BATCH_STEP_EXECUTION`
JobRepository	The record keeper that writes all of the above to the database	Created for you by `@EnableBatchProcessing`
JobLauncher	The object that starts a job	Injected into `BatchTestController`
Tasklet	The simplest kind of step: one piece of code that runs once	The lambda in `helloStep`

Spring Batch: who does what



Spring Batch parts · 5 boxes

The dashed arrow is why the BATCH tables exist: the JobRepository writes every run's status into them.

Why a timestamp parameter. Batch refuses to run a completed job instance again with the same parameters (`JobInstanceAlreadyCompleteException`). Adding `time=System.currentTimeMillis()` makes every run a new instance. For the real upload, the identifying parameter will be the file or batch id.

The hello job. `@EnableBatchProcessing` is one annotation that creates the Batch core beans: `JobRepository`, `JobLauncher`, `JobBuilderFactory`, `StepBuilderFactory` and a transaction manager. It uses your `DataSource` for the `BATCH_*` tables.

```
@Configuration
@EnableBatchProcessing
public class BatchConfig {

    @Bean
    public Step helloStep(StepBuilderFactory steps) {
        return steps.get("helloStep")
                .tasklet((contribution, chunkContext) -> {
                    log.info("Hello from the Spring Batch step");
                    return RepeatStatus.FINISHED;
                })
                .build();
    }

    @Bean
    public Job helloJob(JobBuilderFactory jobs, Step helloStep) {
        return jobs.get("helloJob").start(helloStep).build();
    }
}
```

`RepeatStatus.FINISHED` tells Batch the tasklet is done and should not be called again.

Starting it from the web. Spring Boot would run jobs at startup. Plain Spring does not, so a controller starts the job. This is also how the real upload endpoint will work.

```
@GetMapping("/runhello")
public String runHello() throws Exception {
    JobParameters params = new JobParametersBuilder()
            .addLong("time", System.currentTimeMillis())
            .toJobParameters();
    JobExecution execution = jobLauncher.run(helloJob, params);
    return "Job status: " + execution.getStatus()
            + ", exit code: " + execution.getExitStatus().getExitCode();
}
```

Opening `/epfolearn/runhello` printed `Job status: COMPLETED, exit code: COMPLETED`, and the Tomcat console showed the step's log line.

Tasklet now, chunks later. A tasklet does one thing. The real upload will use chunk-oriented steps: a reader fetches one line, a processor validates it, and a writer saves a group of results. That structure comes in Steps 3 to 5 of the roadmap.

Checking in the database. In SQL*Plus:

```
SELECT job_execution_id, status, exit_code FROM batch_job_execution;
SELECT step_name, status, commit_count FROM batch_step_execution;
SELECT job_name FROM batch_job_instance;
```

You should see one `COMPLETED` row per successful page load.

9. The Batch tables in Oracle, and the ORA-08177 problem

Spring Batch needs its own tables in your schema, and in plain Spring we create them ourselves. Our first run then failed with `ORA-08177`, which taught us what a transaction isolation level is.

Why tables, and why by hand. The JobRepository stores every job, step and parameter in six tables, plus three sequences that hand out ids. Spring Boot would create them automatically. Plain Spring does not, so we ran the script that ships inside `spring-batch-core`.

Step by step, as we did it.

1. Your schema held leftover `BATCH_*` tables and a `BATCH_JOB_INSTANCE_SEQ` sequence from an abandoned Batch 6 experiment. We dropped all nine objects with `DROP TABLE ... CASCADE CONSTRAINTS PURGE` and `DROP SEQUENCE`, then confirmed `user_tables` and `user_sequences` were empty.
2. The script lives inside the jar at `org/springframework/batch/core/schema-oracle10g.sql`. The "10g" is a historical label, and it runs fine on 26ai. We found it with `jar tf spring-batch-core-4.3.8.jar | findstr /i "schema-oracle"`.
3. We extracted it with `jar xf`, copied it to `E:\project\epfo-learn\sql\`, and ran it in SQL*Plus with `@schema-oracle10g.sql`.
4. We checked the result: six tables and the sequences `BATCH_STEP_EXECUTION_SEQ`, `BATCH_JOB_EXECUTION_SEQ` and `BATCH_JOB_SEQ`.

Table	What it stores
`BATCH_JOB_INSTANCE`	One row per job plus its identifying parameters
`BATCH_JOB_EXECUTION`	One row per attempt to run a job (status, start and end times)
`BATCH_JOB_EXECUTION_PARAMS`	The parameters of each run (`KEY_NAME`, `STRING_VAL`, `LONG_VAL` and so on)

Table	What it stores
`BATCH_JOB_EXECUTION_CONTEXT`	Saved state of a job run, used for restart
`BATCH_STEP_EXECUTION`	One row per attempt at a step (read, write and commit counts)
`BATCH_STEP_EXECUTION_CONTEXT`	Saved state of a step, used for restart

The error. Opening `/runhello` returned `CannotSerializeTransactionException ... ORA-08177: can't serialize access for this transaction`. The stack trace showed it inside `SimpleJobRepository.createJobExecution`, so it was in Batch's own bookkeeping. `/hello` and `/dbcheck` still worked, which showed the app and the database connection were healthy.

The cause. A transaction is a group of database changes that succeed or fail together. The isolation level says how strictly a transaction is shielded from other activity. Batch 4.x creates a JobExecution at the strictest level, `SERIALIZABLE`, to stop two copies of the same job starting at once. Oracle enforces that level strictly and can reject the insert with `ORA-08177`. Several reports show this happening even for a single job. We could not pin down exactly why it triggered on your first run, but the documented fix works.

The fix. Build the JobRepository ourselves with a looser level, `READ_COMMITTED`. In Batch 4.3 there is no annotation setting for this, so we provide our own `BatchConfigurer`:

```
public class OracleBatchConfigurer extends DefaultBatchConfigurer {

    private final DataSource dataSource;

    public OracleBatchConfigurer(DataSource dataSource) {
        super(dataSource);
        this.dataSource = dataSource;
    }

    @Override
    protected JobRepository createJobRepository() throws Exception {
        JobRepositoryFactoryBean factory = new JobRepositoryFactoryBean();
        factory.setDataSource(dataSource);
        factory.setTransactionManager(getTransactionManager());
        factory.setIsolationLevelForCreate("ISOLATION_READ_COMMITTED");
        factory.afterPropertiesSet();
        return factory.getObject();
    }
}
```

Then `BatchConfig` registers it as a bean:

```
@Bean
public BatchConfigurer batchConfigurer(DataSource dataSource) {
    return new OracleBatchConfigurer(dataSource);
}
```

A wrong import to remember. `BatchConfigurer` lives in `org.springframework.batch.core.configuration.annotation`, next to `DefaultBatchConfigurer`. A first attempt used `org.springframework.batch.core.configuration`, and the compiler said `cannot find symbol`.

After the fix, `/runhello` returned `COMPLETED`.

10. The validators and unit tests

We wrote the validation rules as plain Java classes first and proved them with tests, before connecting them to Spring Batch. A rule you have tested in seconds is far easier to trust than one buried inside a running job.

The rules. Each validator has one method, `validate(String)`. It returns `null` when the value is fine, or a short reason when it is not. The reason will become the "reason" column of the error file.

Field	Rule	Message when it fails
Aadhaar	Exactly 12 digits	Aadhaar must be exactly 12 digits
Aadhaar	First digit is not 0 or 1	Aadhaar must not start with 0 or 1
Aadhaar	Passes the Verhoeff checksum	Aadhaar checksum is invalid
PAN	Matches `[A-Z]{5}[0-9]{4}[A-Z]`	PAN format is invalid (expected AAAAA9999A)
PAN	4th letter is one of `P,C,H,F,A,T,B,L,J,G`	PAN 4th letter must be one of P,C,H,F,A,T,B,L,J,G
Passport	Matches `[A-PR-WY][1-9][0-9]{6}`	Passport format is invalid (expected a letter and 7 digits, e.g. N1234567)

The Aadhaar checks run in order, cheap ones first: format, then first digit, then checksum. A number that is not even 12 digits never reaches the checksum.

Three rules from the plan are not written yet: field count per line, mobile format and date format. They come when we parse the file.

Regex in plain words. A regular expression describes a text pattern. `[A-Z]{5}` means five capital letters. `[0-9]{4}` means four digits. `[A-PR-WY]` means one letter from A to P, R to W, or Y, which leaves out Q, X and Z. `String.matches` requires the whole text to fit the pattern.

The Verhoeff checksum. The last digit of an Aadhaar number is calculated from the other eleven, so a typo almost always breaks it. Verhoeff catches every single-digit typo and every swap of two neighbouring digits, which a simple digit sum would miss. It only applies to Aadhaar, not PAN or passport.

The method walks the digits from right to left. A running value `c` starts at 0 and is updated using two fixed lookup tables, `D` and `P`. The number is valid if `c` ends at 0. You do not need to understand why the tables work, only that they are standard.

```
public static boolean isValid(String number) {
    if (number == null || number.isEmpty()) return false;
    int c = 0;
    int len = number.length();
    for (int i = 0; i < len; i++) {
        char ch = number.charAt(len - 1 - i);
        if (ch < '0' || ch > '9') return false;
        c = D[c][P[i % 8][ch - '0']];
    }
    return c == 0;
}
```

Unit tests with JUnit 5. A unit test is a small method that calls your code with known inputs and asserts the answers. It runs without Tomcat, so you get feedback in seconds. `@Test` marks a test method, and the `assert...` calls fail the test when the answer is wrong. `mvn test` finds every class under `src\test\java` and runs it.

```
@Test
void wrongChecksumIsRejected() {
    assertEquals("Aadhaar checksum is invalid",
AadhaarValidator.validate("234567890123"));
}
```

The test numbers `234567890124` (valid) and `234567890123` (invalid) were computed with the same algorithm before the tests were written. They are test values, not real people's Aadhaar numbers.

Our result. `mvn test` reported four test classes and `Tests run: 19, Failures: 0, Errors: 0`: 6 for Aadhaar, 4 for PAN, 4 for passport and 5 for Verhoeff.

A trap in the sample data. The sample line in our original design used the PAN `ABCDE1234F`. Its 4th letter is `D`, which is not in the allowed list, so the validator correctly rejects it. Valid sample rows use `ABCPE1234F`. We will keep `ABCDE1234F` as a deliberate bad row to test the error file.

A lesson from the build. Our first `mvn test` said `BUILD SUCCESS` but ran no tests, and we never pinned down exactly why. The next run then failed with cannot find symbol errors, which showed that the main validator file sat in the wrong folder (`src\main\validation` instead of `src\main\java\com\epfo\learn\validation`). Always check that the `T E S T S` section appears and shows `Tests run` greater than zero.

11. Command cheat sheet

Every command we used, grouped by purpose. Enter each as one line, because pasted commands sometimes split across lines.

Purpose	Command	Run it in
Go to the project	`cd /d E:\project\epfo-learn`	Any folder
Confirm you are in the project	`dir pom.xml`	Project folder
Build the WAR (runs tests too)	`mvn clean package`	Project folder
Run only the tests	`mvn test`	Project folder
Clean, then run the tests	`mvn clean test`	Project folder
Remove the old unpacked app	`rmdir /s /q C:\tomcat9\webapps\epfolearn`	Any folder, Tomcat stopped
Remove the old WAR	`del C:\tomcat9\webapps\epfolearn.war`	Any folder, Tomcat stopped
Hand the new WAR to Tomcat	`copy target\epfolearn.war C:\tomcat9\webapps\`	Project folder
See what Tomcat has	`dir C:\tomcat9\webapps`	Any folder
Tell Tomcat where it lives	`set CATALINA_HOME=C:\tomcat9`	The window you start Tomcat in
Start Tomcat in the foreground	`C:\tomcat9\bin\catalina.bat run`	Same window (Ctrl+C stops it)
Find startup errors in the logs	`findstr /i "SEVERE" C:\tomcat9\logs\catalina.*.log`	Any folder
Open SQL*Plus	`sqlplus myuser@127.0.0.1:1521/FREEPDB1`	Any folder (it asks for the password)
List your tables	`SELECT table_name FROM user_tables;`	Inside SQL*Plus

Purpose	Command	Run it in
List your sequences	`SELECT sequence_name FROM user_sequences;`	Inside SQL*Plus
Run the Batch table script	`@schema-oracle10g.sql`	Inside SQL*Plus, started from the `sql` folder
Show a table's columns	`DESC BATCH_JOB_EXECUTION_PARAMS`	Inside SQL*Plus
See job runs	`SELECT job_execution_id, status, exit_code FROM batch_job_execution;`	Inside SQL*Plus
Find a file inside a jar	`jar tf spring-batch-core-4.3.8.jar | findstr /i "schema-oracle"`	The jar's folder under `.m2`
Extract one file from a jar	`jar xf spring-batch-core-4.3.8.jar org/springframework/batch/core/schema-oracle10g.sql`	The jar's folder under `.m2`

The jar lives in `C:\Users\vishal bhat\.m2\repository\org\springframework\batch\spring-batch-core\4.3.8`. Put the path in quotes when you `cd` into it, because of the space in your username.

The web addresses. Tomcat must be running, and the app deployed.

Address	Shows	What it proves
`http://localhost:8080/epfolearn/hello`	Hello from the EPFO learning project!	Tomcat, Spring and your controller work
`http://localhost:8080/epfolearn/dbcheck`	Connected as: MYUSER	The Oracle connection works
`http://localhost:8080/epfolearn/runhello`	Job status: COMPLETED, exit code: COMPLETED	Batch works end to end

After a restart of the laptop. Check Oracle first (`sqlplus` connects), then set `CATALINA_HOME` and start Tomcat. The WAR is still in `webapps`, so no rebuild is needed.

12. Troubleshooting log: problems we hit and how we fixed them

Every error so far came from something simple. The method that found each one: read the first error line and the last `Caused by` line, then check the facts on disk before guessing.

Symptom	Cause	Fix
`/hello` gave 404 right after the logging step, then `/runhello` gave 404 later	`C:\tomcat9\webapps` held no `epfolearn.war` and no `epfolearn` folder, so Tomcat had nothing to serve. We never saw exactly which command was skipped; the build itself was fine	Build, then `rmdir` and `del`, then `copy` last, then `dir C:\tomcat9\webapps` to confirm the WAR is there
Old `BATCH_*` tables in the schema	A leftover Batch 6 experiment	Drop the tables and sequences first, then run the Batch 4.3 script
`/runhello` threw `ORA-08177` in `createJobExecution`	Batch 4.x creates job executions at `SERIALIZABLE` isolation, which Oracle can reject	Custom `BatchConfigurer` with `ISOLATION_READ_COMMITTED` (section 9)
Compile error: `cannot find symbol ... class BatchConfigurer`	Wrong import package	Use `org.springframework.batch.core.configuration.annotation.BatchConfigurer`
`mvn test` printed `BUILD SUCCESS` but no `T E S T S` section	No tests were found or compiled; exact reason not pinned down	Check that the files exist and are saved, run `mvn clean test`, look for `Tests run`
`cannot find symbol ... variable VerhoeffValidator` in the test	The validator file was in `src\main\validation`, which Maven never compiles	Move it to `src\main\java\com\epfo\learn\validation`
After the laptop restarted, nothing answered	Tomcat was stopped and `CATALINA_HOME` was lost with the old window	Check Oracle, set `CATALINA_HOME`, start Tomcat again
`Could not resolve placeholder` at startup	The login file is missing, misnamed or saved as `.txt`	Save `epfolearn.properties` as "All files" in `C:\tomcat9\conf`

Habits that prevent most of these.

- Run `dir C:\tomcat9\webapps` before opening the browser.

- Wait for `BUILD SUCCESS` before touching Tomcat, and for `Server startup in ... milliseconds` before opening a page.
- After `mvn test`, confirm the `Tests run:` line, not just `BUILD SUCCESS`.
- Save every file (look for the unsaved dot on the tab) before building.
- When a Tomcat page shows an error, find the first `SEVERE` line and the last `Caused by` in the console.

13. Glossary

Every term from this document, in plain words, in alphabetical order.

Term	Plain meaning
Annotation	A tag like `@Bean` placed above code to tell a framework what to do with it
Appender	A logging destination, such as the console or a file
Bean	An object that Spring creates and manages for you
Checksum	A value calculated from other digits, used to catch typing mistakes
Chunk	A group of records a Batch step reads, processes and writes together (coming later)
Classpath	The set of folders and jars where Java looks for classes and resource files
Connection pool	A set of ready database connections that code borrows and returns
Controller	A class that answers web requests
DataSource	The object that supplies database connections
Dependency injection	Spring hands each class the objects it needs instead of the class creating them
DispatcherServlet	Spring's single entry servlet that routes each web address to the right method
Environment variable	A named Windows setting that programs can read, such as `JAVA_HOME`

Term	Plain meaning
Isolation level	How strictly a database transaction is shielded from other activity
JAR	A zip of Java classes, used as a library
JDBC	Java's standard way to talk to a database
Job	In Spring Batch, a named recipe made of steps
JobExecution	One attempt to run a job instance
JobInstance	A job plus one set of identifying parameters
JobLauncher	The object that starts a job
JobParameters	The inputs for one job run
JobRepository	Spring Batch's record keeper that stores runs in the `BATCH_*` tables
JUnit	The library used to write and run unit tests
Log level	How serious a log message is: trace, debug, info, warn, error
Maven	The build tool that downloads libraries, compiles, tests and packs the WAR
Package	A named group of Java classes whose name matches its folder path
`pom.xml`	Maven's project file listing libraries and build settings
Regex	A short pattern describing allowed text, such as `[A-Z]{5}`
REST endpoint	A web address that accepts a request and returns data
Sequence	An Oracle object that hands out increasing numbers, used for ids
Servlet	A Java class that handles web requests inside a server like Tomcat

Term	Plain meaning
Staging table	A waiting-area table where rows sit until they are approved
Step	One stage inside a job
Tasklet	A step that runs one piece of code once
Tomcat	The web server that runs your WAR
Transaction	A group of database changes that succeed or fail together
Unit test	A small method that checks one piece of code with known inputs
Verhoeff	The checksum method used for Aadhaar numbers
WAR	A zip of a web app (`WEB-INF\classes` and `WEB-INF\lib` plus web files) that Tomcat runs
`web.xml`	The old XML file that described a web app to Tomcat; we use `AppInitializer` instead
`javax` and `jakarta`	Two package-name families for the servlet API; Spring 5.3 and Tomcat 9 use `javax`

14. Roadmap: what comes next

The foundation is finished, so the remaining work is the real upload feature, built in small tested steps. The numbers below follow your original plan list. The step numbers we used in chat during setup count differently, so go by the topic names.

Plan step	Topic	What you will learn
3	Read the file with `FlatFileItemReader` and `DelimitedLineTokenizer` (separator `#~#`) into an `EmployeeRecord`	How a Batch reader turns text lines into Java objects; JSAPAR as the work-style alternative later

Plan step	Topic	What you will learn
4	`ItemProcessor` that runs the validators and collects every error per line	How the processor sits between reader and writer; field count, mobile and date checks join the validators
5	`JdbcBatchItemWriter` into the staging table	Chunk steps, batched SQL inserts, creating `upload_batch` and `employee_staging`
6	Error file plus conditional flow between steps	The all-or-nothing rule: if any error exists, write the error file and skip the load
7	REST upload API (multipart), JobParameters, launching with `JobLauncher`, status and error-file download	Accepting a file over HTTP and starting a job per file
8	Approval endpoint and Job 2: staging to master with `JdbcCursorItemReader`	Reading database rows as a stream; duplicate check on Aadhaar
9	Oracle stored procedure, package and cursor version of the insert, called with `SimpleJdbcCall`	How the real project's PL/SQL side fits in
10	Restart and failure handling demo	Why Batch records every step: resuming a failed job from where it stopped

Multithreading comes much later, after the single-threaded version works.

Good moments for new notes. Before plan step 7 (REST upload) and before plan step 9 (stored procedures), because each adds many moving parts.

15. Learning resources

The official pages below were checked on 29 October 2026. Most were opened directly. The Spring Batch chapter links come from its opened reference index, and the schema appendix was read in search results. Videos and one Oracle reference were found by search only, and the list says which.

A caution about tutorials. Most beginner Spring Batch videos and courses use Spring Boot, which sets up much of what we wrote by hand (the `DataSource`, the Batch tables, starting jobs). Newer Batch versions also changed the builder style. Use them to learn the concepts (job, step, reader, processor, writer), and keep following our code for the exact classes, because ours is plain Spring with Batch 4.3.

Official documentation (checked)

What we did	Read this	Why
Maven builds (section 4)	Maven in 5 Minutes	Explains the POM, goals versus lifecycle phases, and why `mvn package` also runs the earlier phases
Deploying a WAR (section 4)	Tomcat 9 web application deployment	Describes copying a WAR into `webapps`, and what `autoDeploy` and `unpackWARs` do
Spring startup and requests (section 5)	Spring Framework 5.3.26: Web on Servlet Stack	Section 1.1 covers `DispatcherServlet`, the context hierarchy and `WebApplicationInitializer`, our exact version
Job, Step, JobInstance (section 8)	The Domain Language of Batch	The official explanation of the terms in our table
Starting jobs (section 8)	Configuring and Running a Job	Covers `JobLauncher`, `JobRepository` and `@EnableBatchProcessing`
The Batch tables (section 9)	Meta-Data Schema appendix	Describes every column of the six tables and the sequences
Unit tests (section 10)	JUnit 5 User Guide	Section 2 covers `@Test` and assertions. This is version 5.14.0, newer than our 5.10.3, and the basics are the same
Verhoeff checksum (section 10)	Verhoeff algorithm on Wikipedia	Says it detects all single-digit errors and all swaps of adjacent digits, and lists Aadhaar among its uses

The whole Spring Batch 4.3 reference is worth keeping open. Its chapters on configuring a step, reading and writing items, and item processing match plan steps 3 to 5.

Courses

Course	Platform	How it fits
Let's begin with Spring Batch Framework (Black Hills)	Udemy	A job reads patient records from a comma-delimited file, processes them, writes them to a database, and tests each part. It uses Spring Boot, and the page shows it was last updated in 9/2021. Closest match to plan steps 3 to 5
Oracle PL/SQL Programming: Apply, Build & Optimize (EDUCBA)	Coursera	Beginner level, about 7 hours. Covers blocks, cursors, records and exception handling, which is the groundwork for plan step 9. I read its description page, not the lessons

Prices and availability change, so check them on the platform.

Videos found by search (not opened)

I could not open YouTube pages from here, so I have not watched these, and I do not know their quality, length or date. The titles are as the search listed them. Check the upload date and the comments before investing time.

Topic	Video title as listed	Link
Spring Batch concepts	Spring Batch for Beginners - Introduction to Spring Batch	watch
Spring Batch concepts	Spring Batch Tutorial for Beginners	watch
Maven	Master Apache Maven in one video	watch
Maven lifecycle	Maven Lifecycles and Basic maven Commands	watch
Spring MVC without `web.xml`	NO WEB.XML ? || Spring MVC Java based configuration || WebApplicationInitializer || Part 2	watch

Topic	Video title as listed	Link
Spring MVC without `web.xml`	Configure Dispatcher Servlet in just two lines of code. How does this magic happen? || Spring MVC	watch
JUnit 5	JUnit 5 Tutorial For Beginners	watch
JUnit 5	JUnit Tutorial (Unit Testing in Java) - JUnit 5 Framework (playlist)	open

One more reference, found by search and not opened. Oracle's PL/SQL Language Reference, 19c edition (PDF) is a large reference, not a tutorial. Your database is 26ai, and the PL/SQL basics there are the same.

Searching on your own. Add the version to your search, such as "Spring Batch 4" or "Spring 5", and the words "Java config" or "no Spring Boot". That finds material closer to our stack.